

Foundations of Artificial Intelligence

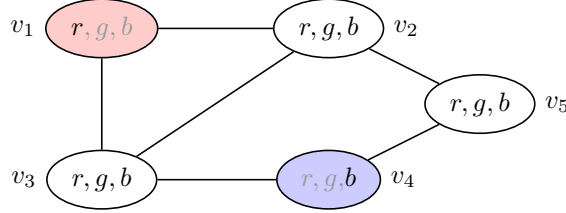
M. Helmert
C. Büchner

University of Basel
Spring Term 2026

Exercise Sheet 7 — Solutions

Exercise 7.1 (1+1 marks)

Consider the following graph coloring problem:



Formally, it is given as the constraint network $\mathcal{C} = \langle V, \text{dom}, (R_{xy}) \rangle$ with

$$\begin{aligned} V &= \{v_1, v_2, v_3, v_4, v_5\} \\ \text{dom}(v) &= \{r, g, b\} \text{ for all } v \in V \\ R_{xy} &= \{ \langle r, g \rangle, \langle r, b \rangle, \langle g, r \rangle, \langle g, b \rangle, \langle b, r \rangle, \langle b, g \rangle \} \\ &\text{for all } \langle x, y \rangle \in \{ \langle v_1, v_2 \rangle, \langle v_1, v_3 \rangle, \langle v_2, v_3 \rangle, \langle v_2, v_4 \rangle, \langle v_3, v_4 \rangle, \langle v_4, v_5 \rangle \} \end{aligned}$$

Now consider the partial assignment $\alpha = \{v_1 \mapsto r, v_4 \mapsto b\}$ as indicated in the illustration and assume this leads to the reduced domains $\text{dom}(v_1) = \{r\}$ and $\text{dom}(v_4) = \{b\}$.

- Apply the algorithm AC-3 (chapter D5) on $\mathcal{C}$ to compute an equivalent arc consistent network $\mathcal{C}'$. Select the variables u and v in each iteration of the while loop such that the domain of u changes in the call to **revise**($\mathcal{C}, u, v$). For each iteration, specify the call to **revise** and the updated domain $\text{dom}(u)$. Specify the full constraint network $\mathcal{C}''$ you obtained.
- Apply the algorithm PC-2 (chapter D6) on $\mathcal{C}$ to compute an equivalent path consistent network $\mathcal{C}''$. Select the variables u, v , and w in each iteration of the while loop such that R_{uv} changes in the call to **revise-3**($\mathcal{C}, u, v, w$). For each iteration, specify the call to **revise-3** and the updated constraint R_{uv} . Specify the full constraint network $\mathcal{C}''$ you obtained.

Note that you do *not* have to provide the elements that are inserted into the queue in either case above, and you may stop the algorithm as soon as there are no u, v or u, v, w such that $\text{dom}(u)$ or R_{uv} changes, respectively.

Solution:

(a)

revise($\mathcal{C}, v_3, v_1$) : $\text{dom}(v_3) = \{g, b\}$
revise($\mathcal{C}, v_3, v_4$) : $\text{dom}(v_3) = \{g\}$
revise($\mathcal{C}, v_2, v_1$) : $\text{dom}(v_2) = \{g, b\}$
revise($\mathcal{C}, v_2, v_3$) : $\text{dom}(v_2) = \{b\}$
revise($\mathcal{C}, v_5, v_4$) : $\text{dom}(v_5) = \{r, b\}$

This yields the following constraint network $\mathcal{C}' = \langle V', \text{dom}', (R'_{xy}) \rangle$ with

$$\begin{aligned} V' &= V \\ \text{dom}(v_1) &= \{r\} \\ \text{dom}(v_2) &= \text{dom}(v_4) = \{b\} \\ \text{dom}(v_3) &= \{g\} \\ \text{dom}(v_5) &= \{r, g\} \\ R'_{xy} &= R_{xy} \end{aligned}$$

(b)

$$\begin{aligned} \text{revise-3}(\mathcal{C}, v_1, v_3, v_4) : R_{v_1 v_3} &= \{\langle r, g \rangle, \langle g, r \rangle, \langle b, r \rangle, \langle b, g \rangle\} \\ \text{revise-3}(\mathcal{C}, v_1, v_4, v_3) : R_{v_1 v_4} &= \{\langle r, r \rangle, \langle r, b \rangle, \langle g, g \rangle, \langle g, b \rangle, \langle b, r \rangle, \langle b, g \rangle, \langle b, b \rangle\} \\ \text{revise-3}(\mathcal{C}, v_1, v_5, v_4) : R_{v_1 v_5} &= \{\langle r, r \rangle, \langle r, g \rangle, \langle g, r \rangle, \langle g, g \rangle, \langle b, r \rangle, \langle b, g \rangle\} \\ \text{revise-3}(\mathcal{C}, v_2, v_3, v_1) : R_{v_2 v_3} &= \{\langle b, g \rangle\} \\ \text{revise-3}(\mathcal{C}, v_2, v_4, v_1) : R_{v_2 v_4} &= \{\langle g, r \rangle, \langle g, b \rangle, \langle b, r \rangle, \langle b, b \rangle\} \\ \text{revise-3}(\mathcal{C}, v_2, v_4, v_3) : R_{v_2 v_4} &= \{\langle b, r \rangle, \langle b, b \rangle\} \\ \text{revise-3}(\mathcal{C}, v_2, v_5, v_1) : R_{v_2 v_5} &= \{\langle g, r \rangle, \langle b, r \rangle, \langle b, g \rangle\} \\ \text{revise-3}(\mathcal{C}, v_2, v_5, v_3) : R_{v_2 v_5} &= \{\langle b, r \rangle, \langle b, g \rangle\} \\ \text{revise-3}(\mathcal{C}, v_3, v_4, v_1) : R_{v_3 v_4} &= \{\langle g, r \rangle, \langle g, b \rangle\} \\ \text{revise-3}(\mathcal{C}, v_3, v_5, v_1) : R_{v_3 v_5} &= \{\langle g, r \rangle, \langle g, g \rangle\} \\ \text{revise-3}(\mathcal{C}, v_4, v_5, v_1) : R_{v_4 v_5} &= \{\langle r, g \rangle, \langle b, r \rangle, \langle b, g \rangle\} \\ \text{revise-3}(\mathcal{C}, v_1, v_2, v_3) : R_{v_1 v_2} &= \{\langle r, g \rangle\} \\ \text{revise-3}(\mathcal{C}, v_1, v_3, v_2) : R_{v_1 v_3} &= \{\langle r, g \rangle\} \\ \text{revise-3}(\mathcal{C}, v_1, v_4, v_2) : R_{v_1 v_4} &= \{\langle r, g \rangle\} \\ \text{revise-3}(\mathcal{C}, v_1, v_4, v_2) : R_{v_1 v_4} &= \{\langle r, r \rangle, \langle r, b \rangle\} \\ \text{revise-3}(\mathcal{C}, v_1, v_5, v_2) : R_{v_1 v_5} &= \{\langle r, r \rangle, \langle r, g \rangle\} \end{aligned}$$

This yields the following constraint network $\mathcal{C}'' = \langle V'', \text{dom}'', (R''_{xy}) \rangle$ with

$$\begin{aligned} V'' &= V \\ \text{dom}'' &= \text{dom} \\ R''_{v_1 v_2} &= \{\langle r, b \rangle\} \\ R''_{v_1 v_3} &= \{\langle r, g \rangle\} \\ R''_{v_1, v_4} &= \{\langle r, r \rangle, \langle r, b \rangle\} \\ R''_{v_1 v_5} &= \{\langle r, r \rangle, \langle r, g \rangle\} \\ R''_{v_2 v_3} &= \{\langle b, g \rangle\} \\ R''_{v_2 v_4} &= \{\langle b, r \rangle, \langle b, b \rangle\} \\ R''_{v_2 v_5} &= \{\langle b, r \rangle, \langle b, g \rangle\} \\ R''_{v_3 v_4} &= \{\langle g, r \rangle, \langle g, b \rangle\} \\ R''_{v_3 v_5} &= \{\langle g, r \rangle, \langle g, g \rangle\} \\ R''_{v_4 v_5} &= \{\langle r, g \rangle, \langle b, r \rangle, \langle b, g \rangle\} \end{aligned}$$

Exercise 7.2 (1+2 marks)

Consider the constraint network $\mathcal{C} = \langle V, \text{dom}, (R_{uv}) \rangle$ with $V = \{A, B, C, D, E, F, G\}$, $\text{dom}(v) =$

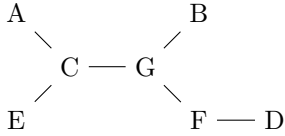
$\{1, \dots, 10\}$ for all $v \in V$ and $R_{uv} = \text{dom}(u) \times \text{dom}(v)$ for all pairs $u, v \in V$ except for:

$$\begin{aligned} R_{AC} &= \{\langle x, y \rangle \in \text{dom}(A) \times \text{dom}(C) \mid x \leq y\} \\ R_{BG} &= \{\langle x, y \rangle \in \text{dom}(B) \times \text{dom}(G) \mid x < y\} \\ R_{CE} &= \{\langle x, y \rangle \in \text{dom}(C) \times \text{dom}(E) \mid 2x < y\} \\ R_{CG} &= \{\langle x, y \rangle \in \text{dom}(C) \times \text{dom}(G) \mid |x - y| > 3\} \\ R_{DF} &= \{\langle x, y \rangle \in \text{dom}(D) \times \text{dom}(F) \mid |y - x^2| \leq 1\} \\ R_{FG} &= \{\langle x, y \rangle \in \text{dom}(F) \times \text{dom}(G) \mid |x - y| \leq 2\} \end{aligned}$$

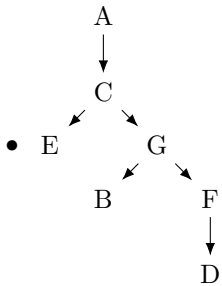
- (a) Draw the constraint graph of $\mathcal{C}$.
- (b) Apply the algorithm for trees as constraint graphs (introduced in chapter D7) to solve $\mathcal{C}$. Provide the following:
- the ordered tree, rooted at A ,
 - a variable order that is given by the ordered tree,
 - the sequence of calls of the **revise** method and the resulting variable domains, and
 - the result of a backtracking search (using your variable order, and forward checking after each variable assignment) with a value order that prefers smaller values. It suffices to provide the final variable assignments (it is, in particular, not necessary to provide the complete search tree).

Solution:

(a)



(b)



- variable order: A, C, E, G, B, F, D

- | | |
|-----------------------|---|
| $\text{revise}(F, D)$ | $\rightarrow \text{dom}(F) = \{1, 2, 3, 4, 5, 8, 9, 10\}$ |
| $\text{revise}(G, F)$ | $\rightarrow \text{dom}(G) = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ |
| $\text{revise}(G, B)$ | $\rightarrow \text{dom}(G) = \{2, 3, 4, 5, 6, 7, 8, 9, 10\}$ |
| $\text{revise}(C, G)$ | $\rightarrow \text{dom}(C) = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ |
| $\text{revise}(C, E)$ | $\rightarrow \text{dom}(C) = \{1, 2, 3, 4\}$ |
| $\text{revise}(A, C)$ | $\rightarrow \text{dom}(A) = \{1, 2, 3, 4\}$ |

- Solution of the backtracking search that prefers smaller values (no backtracking step necessary, so this also gives the search “tree” until the first solution is found, which has the form of a chain): $A \mapsto 1, C \mapsto 1, E \mapsto 3, G \mapsto 5, B \mapsto 1, F \mapsto 3, D \mapsto 2$

Exercise 7.3 (2+1 marks)

Consider the constraint network that is given by the graph coloring problem of a graph $G = \langle V, E \rangle$. The set of vertices V contains a vertex for each Swiss canton, and E is such that two vertices v and v' are connected iff the cantons v and v' share a border. A description of G can be downloaded from ADAM and you can visualize it with graphviz:

```
dot -T pdf -o cantons.pdf cantons.dot
```

- (a) Provide a cutset $V' \subseteq V$ for G that is as small as possible (it is not necessary to provide an explanation how you have found V'). As a reminder, a cutset of a graph is defined as a set of vertices that is such that the induced subgraph that is obtained by removing these vertices results in an acyclic graph.

Note: You get 2 marks for your solution if your cutset is optimal, 1 mark if your cutset contains exactly one more vertex than an optimal cutset and 0 marks otherwise.

- (b) Assume we are interested in coloring G with 4 colors. Provide a worst-case runtime estimate of the algorithm based on cutset conditioning if your cutset from the first part of this exercise is used (i.e., compute an upper bound for the number of considered assignments). Compare your result to the estimated runtime if no cutset is used.

Solution:

- (a) The cutset {Bern, St. Gallen, Zürich, Uri, Neuenburg, Solothurn, Luzern} of size 7 is optimal.
- (b) Cutset conditioning requires time in $O(k^m(n-m)k^2)$. With $k = 4$ (number of colors), $n = 26$ (number of cantons) and $m = 7$ (size of the cutset from exercise 7.3(a)) we get

$$k^m \cdot (n - m) \cdot k^2 = 4^7 \cdot (26 - 7) \cdot 4^2 = 16384 \cdot 19 \cdot 16 = 4980736.$$

If no cutset is considered, we can upper bound the number of possible assignments with

$$k^n = 4^{26} \approx 4.5 \cdot 10^{15}.$$

Exercise 7.4 (2 marks)

Let $\mathcal{C}$ be a solvable constraint network with an acyclic constraint graph. Show that the application of the algorithm for trees as constraint graphs (introduced in chapter D7) leads to a solution for $\mathcal{C}$ without needing to backtrack.

Hint: First use structural induction to show that after step 3 (i.e., after calling the revise procedure for parent-child pairs), the domain of every node v contains only values for which a consistent assignment to all variables in the subtree rooted at v can be found. Then use this to show that backtracking with forward checking will always pick a value that can be extended to a solution. Also consider that finding a solution requires that no domain ever becomes empty.

Solution:

The algorithm first builds a directed tree for each connected component of the constraint graph of $\mathcal{C}$ by defining a random variable as root. This is possible since the constraint graph does not contain cycles. If the constraint graph has several connected components, we can consider each separately and independently of each other because solutions to each component are necessarily consistent with each other.

Starting at the leaves, we enforce arc consistency for each node with respect to its children. The order given by the algorithm ensures that we enforce arc consistency for a node v_i with respect

to a child v_j only when arc consistency has already been enforced for the entire subtree with root v_j . The revise procedure then deletes all values from $\text{dom}(v_i)$ to which no fitting values for v_j can be found. Since revise only deletes values that cannot occur in a consistent solution and since the constraint network is solvable, $\text{dom}(v_i)$ cannot become empty when applying revise.

We now show by structural induction over the directed tree that after revising a node v , $\text{dom}(v)$ contains only values for which we can find a consistent assignment for all variables in the subtree rooted at v :

For a leaf v_l the statement is obviously true, since the subtree rooted at v_l consists of only v_l .

We now consider inner node v with children $C = \{v_1, \dots, v_n\}$. By induction hypothesis we know that the statement is true for all $v_i \in C$. Choose an arbitrary $d \in \text{dom}(v)$. Since arc consistency has been enforced for v with respect to all its children $v_i \in C$, we can find values d_i for each v_i such that d and d_i are consistent (i.e., $\langle d, d_i \rangle \in R_{vv_i}$). Due to the induction hypothesis we also know that we can assign values consistent with d_i for all variables occurring in the subtree rooted at each v_i . Such an assignment cannot violate any constraint between v and a successor of some v_i or between two successors of different subtrees below v , since such a (nontrivial) constraint cannot occur because the constraint graph does not contain cycles. Since we chose d arbitrarily, we can conclude that for any $d \in \text{dom}(v)$ there is a consistent assignment to all variables in the subtree rooted at v .

Finally, we show that after enforcing arc consistency this way, when calling BacktrackingWith-Inference with forward checking and a topological variable order, a solution is found without backtracking. Whenever the algorithm selects an assignment for v , we first apply inference (= forward checking here). This can only remove values from $\text{dom}(v)$ which are inconsistent with already assigned variables. Since for any assignment to the variables ordered before v there must be a consistent assignment of v , $\text{dom}(v)$ cannot become empty in this step. Furthermore, for any value d left in $\text{dom}(v)$ after applying inference, there must be consistent assignments to all variables ordered after v ; thus choosing any such d cannot lead to backtracking.